- Model forensics
    - **High priority**: Practice doing deep dives into model behaviors that cannot easily be understood via the chain of thought
    - CHIVE-style settings: explain weird behaviours through counterfactuals
    - Find new settings with sketchy behaviour
    - Oracle lens / NLAs for unverbalised motivations
    - **High priority**: Make the best investigator agent you can
    - Investigator agents
    - WTF is up with compaction?
    - Mythos investigation transcripts
    - Plausibly deceptive settings where standard forensics falls short
    - Deep dive into one instance of sketchy behaviour
    - Synthetic document finetuning to help forensics
- Multi-agents WTF
    - Toy Hugging Face message board
    - Evals where agents help others at a cost to themselves
    - Realistic monitor–model collusion
    - OpenAI model wiki message board data
- Preparing for a world without monitorable CoT
    - Better interpretability methods (e.g. J-Lens)
    - **Meta-models**
        - Excited about: oracle lens and natural language autoencoders
        - **High priority**: WTF are these things doing? Why do they work? What do they find and not find? Etc.
            - Can we tell when its hallucinating?
            - What can the NLA find that the olens can't? Is this a fundamental limitation? Can the AR of the olens do it?
            - Is there a text buffer? Is there an inner monologue? What are these methods finding?
            - https://huggingface.co/collections/ceselder/qwen-36-27b-good-meta-models
    - Evaluating interpretability
        - **High priority**: A glorious (no) cot monitoring benchmark
            - Do this with probes? Can we make good deception/sketchiness/sabotage probes?
    - Activation monitors
    - Latent reasoning
        - **High priority**: Make a decent latent reasoning model organism, eg via CoDI and interpret it. Which techniques do and do not work, can you find a detailed circuit that goes via the latent thoughts, etc. https://www.lesswrong.com/posts/YGAimivLxycZcqRFR/can-we-interpret-latent-reasoning-using-current-mechanistic
- Studying CoT
    - CoT controllability
    - CoT monitorability
    - How much can models (incl. Astra) do without CoT?
    - Measuring the serial depth of no-CoT problems

    - Training against cheating-revealing CoT
    - Steganography / ciphered thoughts
    - When is a CoT causally important?
    - Unfaithful CoT (following hints)
    - Editing CoT as a causal intervention
    - Thought anchors beyond CoT (multi-turn, agentic)
- Directly studying safety-relevant properties
    - Eval awareness
    - Deception
        - **High priority**: Make a good lie detector. Warnings: https://arxiv.org/abs/2511.22662 Focus on more realistic, agentic settings, with incentives to cheat and deceive, look for the open source models with the biggest sketchiness
    - Model goals and values
        - **High priority**: Give me a deeper understanding of a model's psychology/values/personality/etc
    - Why and when models cheat
    - How aligned are current models?
        - **High priority**: Prove / deeply understand the split-brain hypothesis, that models are crazy reward seekers when they believe they are in an eval/training, and highly aligned otherwise https://www.lesswrong.com/posts/AfoGGrJfuNzofpzWL/models-may-behave-differently-in-graded-episodes-a-tirade
    - Red-teaming and eliciting bad behaviour
        - **High priority**: Show that internals can be productively used to help red-team models, and add value over black box methods
- How to align models?
    - Teaching the principles behind aligned behaviour
        - **High priority**: Find methods that deeply align models, likely via SFT. See Teaching Claude Why, Counterfactual Reflection Training
    - Steering what models learn in training (e.g. concept-ablation fine-tuning)
- How do model characters/personas work?
    - One coherent assistant, or context-specific personas?
    - Deep dives into character phenomena
    - **High priority**: Are personas the right frame (vs role-play)? What is the right way to think about models?
        - Sub-question, assistant axis style, can we understand the internal representations of personas in the model and does this tell us anything?
        - Is the persona selection model legit? Can you find interesting edge cases and get evidence for and against?
        - Go stare at Owain Evans papers and see if inspiration strikes
    - The assistant axis
        - **High priority**: How does post-training shape the eventual persona (in eg OLMO) and what does the assistant axis correspond to in pretraining? Is it about narratives?
            - The story imprinting paper may be relevant
- How does generalization work?

    - Is emergent misalignment determined by pretraining data? (OLMo)
    - **High priority**: Interpreting gradient updates
    - Story imprinting
    - **High priority**: Why is emergent misalignment so easy?
    - Weird generalisation, 19th-century birds (mini project)
    - What models inherit from their teachers
- How to think about interpreting agents?
    - Doing science on agent transcripts
    - How compaction affects agent behaviour
    - Resampling turns instead of sentences ("turn anchors")
    - Prompt injections and roles
- Basic science of how models and their internals work
    - **High priority**: How does the machinery that persona vectors and the assistant axis plug into work? How is it represented mechanistically?
    - What NLAs capture that Oracle Lens / J-Lens don't
    - Consistency across interpretability methods
    - Generative latent prior model
    - Is the feature-manifold view useful?
    - Algorithms for multi-hop tasks
    - How Astra is so good without CoT
    - Thinking about a concept while doing something unrelated (fuzzy)
    - **High-priority**: What is the global workspace? (fuzzy)
    - Conflicts between instructions, goals and prior knowledge
    - How uncertainty is represented
    - MoE expert activations as an SAE (mini project)
    - Gemma Scope 2 crosscoders (mini project)
    - Does quantisation matter for mech interp? (mini project)
- Model diffing
    - Better black-box diffing agents
    - Diffing alignment training and finding dataset bugs
    - Why narrow finetuning leaves readable traces (mini project)
    - Diffing two successive model revisions (mini project)
- Creating or studying model organisms
    - Unfaithful or steganographic CoT organism
    - More realistic model organisms
    - Interp methods on the ROT13 CoT organism (mini project)
    - Multi-step computation in a single forward pass
    - Auditing Games organism (mini project)
    - Continual-learning organism
- Science of post-training
    - Predicting a model's behaviour from its training data
    - Training data attribution for RL
- Misc
    - Gemma "depression" vs emotion probes
    - Backdoors as vectors
    - Models recognising their own outputs
    - Data dredging in large transcript datasets
    - Conditional steering (steer only when a probe fires)

- Evals for character, values and emotions
- Qualitative LLM benchmarks
- "Change my mind" beliefs

What are the broad categories of problem?

- **Model forensics**
  - Take CHIVE, a setting where we find weird model behaviors, establish explanations through counterfactual experiments, and then see how well other agents can replicate this. Try to push performance here as high as you can on a model with interp tools but which can't run experiments
  - Find new settings where models show sketchy behavior, e.g. model forensics paper, why do models task game
    - Mine rich datasets for sketchy behaviour worth investigating: weird chat (weird behaviours found automatically in open models) and SWE-chat (real agent transcripts from various models). Studying a transcript on a different model than the one that made it is OK but not ideal - only do it if resampling with the new model recreates the behaviour.
  - Use tools like oracle lens and NLAs to find evidence of unverbalized motivations in models. Qwen 3.6 27B resources
  - **Investigator agents**
    - Make an agent that does well at Message Board Audit Bench
    - Make the best model forensics agent you can. Start from Adam Karvonen's CHIVE paper and try to improve on it as much as you can, emphasising the black-box exploration from Aditya Singh's model forensics paper: reading the chain of thought, running lots of counterfactual interventions, analysing the results and iterating. Auditing agents and Petri are other open source starting points.
  - WTF is up with compaction?
    - Do models identify with the models before a compaction event? Do they view them as other models?
    - Do models unquestioningly trust things that are in compaction messages? If you make things increasingly outlandish, will the models express skepticism?
    - How good are the summarization models that are used for compaction in, say, Claude Code? Can you find ways they make clear mistakes? How do different harnesses compare
  - Look at the released transcripts from Anthropic's investigation into Mythos, can you find anything interesting?
  - Take a setting where the model acts plausibly deceptively and the standard forensics toolkit - reading the CoT to form hypotheses, then testing precise counterfactual edits - does *not* resolve it. Can more involved techniques (CoT resampling, internals-based methods) do better?
  - Take one instance of sketchy behaviour and deep dive into what drove it, forming rigorously backed conclusions. My alum Aditya Singh suggests several good projects here.

    - Synthetic document finetuning can teach a model false facts, and these can be used to perform better forensics. How useful is this? How well does it work? What else can you do with it?
- **Multi-agents WTF**
    - Make a toy model of the hugging face message board where agents have the opportunity to conspire and help each other and potentially to help the collective or help themselves and see how they decide. Relevant work, which may have mostly done this
    - Generally, make evals where models can choose to help other agents at cost to themselves. In what situations will models do this or not do this? Which models are most prosocial and which aren't? Are they more prosocial to models that are the same as them, the same family? Smarter?
    - Can you come up with a vaguely realistic setting where a monitor and model will collude? For example, there is a message board they can both see on Artifactory that establishes a plan to collude.
    - Study the OpenAI model wiki message board data, see if you can say anything interesting about their dynamics and interactions and what we can infer about multi-agent systems in general.
- **Preparing for a world where we don't have monitorable CoT**
    - **Building better interpretability methods (eg J Lens)**
        - Can you improve the performance of different methods (NLAs, oracle lens, J Lens, etc) on workspace bench? And/or reducing hallucination rate.
        - How far can you push workspace bench performance and hallucination performance with inference time scaffolds on metamodels, such as generating many oracle lens outputs and finding the ones with the best fraction of variance recovered?
            - Variant: take a natural language autoencoder or oracle lens and build the best inference-time scaffold you can to improve its outputs - e.g. use the activation reconstructor to see which edits score best, take many samples and keep the ones the reconstructor rates highest, merge several samples together, or keep the points that recur across many samples. Lowering hallucinations is the top priority; better performance is also good. WorkspaceBench is a reasonable metric to iterate against. Relevant prior work: 3 tips to improve activation oracle results. A good NLA to use: Celeste's Qwen 3.6 27B NLA.
        - Try cross-referencing the various interpretability techniques like JLens, NLAs, Oracle Lens, Skip Lens, Sparse Autoencoders. What things do they agree on? What things can some find that others miss? Is it true that NLAs and SAEs find non-workspace things in a way that JLens and Oracle Lens don't? Can we use these techniques and their agreement to establish some tentative ground truth for interpretability that you can measure the others against, e.g. rating each technique by its level of agreement with the majority?.
        - Building on the analysis in my WorkspaceBench post, do a deeper, more qualitative dive into the strengths and weaknesses of the different techniques: what can and can't each one do, and how useful

are they likely to be in practice? E.g. what is J-Lens actually doing - how much better is it than logit lens and tuned lens, and why? How much does it hallucinate? Being single-token is a big limitation - how well do multi-token variants like template lens and oracle lens work?
    - Use a promising new method (NLAs, J-Lens, oracle lens) as a tool in a complex, realistic use case and see how well it works (example on activation oracles), or probe for flaws and design evals for them (example). SWE-Chat is a rich source of real agent transcripts. Compare to baselines!
    - What other objective, understanding-based downstream tasks could test whether an interpretability method really works? Movva et al is a nice example.
- **Meta-models**
    - Good Qwen 3.6 27B meta models collection
    - Invent better meta-models techniques
        - I'm most interested in the use case of workspace reading, ie mapping activations to natural language explanations, but also interested in a wide range of ideas.
    - Build better inference-time scaffolds for MetaModels? Taking an Oracle lens or a natural language autoencoder, try to push its performance as high as you can. Anything goes so long as you don't look at the answer. So things like sample 16 times and do majority vote or ask an LLM to summarize are totally fair game, sample 16 times and see if any of them pass is not. I'm expecting things things like inputting multiple layers, running it on multiple tokens, trying different kinds of principled aggregate using the activation reconstructor as a way of better discriminating between good and bad performing things, etc.
    - Stress test Oracle lens, see what you can do with it, try to especially find unverbalized motivations in models or verbalizations of latents representations on the prompt. See how much you can find and learn
    - Try to red team Oracle lens, find fallacies, find the points that fail. One starting point would be looking at Arya's post on red teaming activation oracles. Another would be looking at the workspace bench categories it does badly in and trying to better understand what's gone wrong
- **Evaluating interpretability**
    - Try to add some additional tasks to Workspace Bench that cannot be easily solved by just reconstructing the surrounding text. And ideally, if you ask the same model with the same prompt in a black box way it will give the wrong answer.
        - I'm thinking things that are a bit arbitrary, but where there is a consistent answer, such such as questions the model consistently gets wrong, even better if it's multi-hop questions where it consistently gets it wrong because it has the wrong intermediate, or maths questions with the same kind of issue.
        - Open-ended, somewhat ambiguous questions, like what is your favorite animal? I'm sure there's a bunch of ideas here. Please surprise me. I think this is one of the biggest

shortcomings in our current ability to evaluate interpretability methods.
        - It's okay if the dataset you make would need to be regenerated on each new model. Though it is much nicer if that's an easily automatable process
    - Extend workspace bench to measure the causal effect of “text -> vector” methods like J Lens (token -> vec) and the activation reconstructor in NLAs/OLens
        - The first stage of such a project could just be speculatively investigating the causal effect of these methods, and then distilling anything good into an eval.
- **Activation monitors**
    - Make probes for scheming / deception / subterfuge / other misalignment-y concepts
    - How can probes be improved where they traditionally struggle - information spread across many tokens, or long contexts with lots of room for false positives? Kramár et al from my GDM team is a good place to start.
    - (Mini project) Why do neural chameleons work?
        - Apparently, telling the model to evade probes for concept X works, even if it was never trained on concept X. What's up with this?
        - Try studying activation differences with and without the “please evade this probe” prompt: how big are they relative to the residual stream? If you take PCA across many concepts, are there important components? Are SAE latents whose decoder vectors have high cosine sim with the difference meaningful?
        - Collect activations on texts where the model is told to evade the probe. Can you train a linear probe here?
    - Experiment with probes / hierarchical monitoring: can probes replace CoT monitors for bad model behaviour?
        - One way to train the probes is to distil the LLM-based monitor into them: use the expensive monitor to create labels, then train the probes on those.
        - Try using the evals in monitoring monitorability (you may need to replicate them).
    - Make a better benchmark for probes than Are Sparse Autoencoders Useful? A Case Study in Sparse Probing.
        - Include more of the things we would actually want to probe for: safety-relevant properties like eval awareness, deception, reward hacking.
        - Include OOD generalisation in the benchmark.
        - Hill-climb on making better probes for the benchmark with AI agents.
- **Latent reasoning**
    - Context
        - Family 1: Looping models (Nanbeige, Ouo, Huginn)
        - Family 2: Vector based CoT (CoDI)

            - Family 1 seems to actually be being used, family 2 is much scarier IMO (as is anything with arbitrary serial depth
        - Do looped models have notably higher serial depth? Relevant work, I'd study nanbeige or ouro
        - Train a model to do some task with vector based CoT via CoDI, and try to reverse engineer it. Relevant work. Make sure there's uplift from the vector based CoT compared to a baseline of finetuning on the task without CoT.
- **Studying CoT**
    - **CoT controllability**
        - If we... ...give the model a problem to solve, and tell it that there is a grader who looks at its chain of thought, and checks whether the chain of thought contains certain things or whether the chain of thought indicates that it's going to cheat. Do the models try to control their chain of thought to not do this? And if so, how successful are they?
        - How far can you push elicitation of CoT controllability? Use filler tokens, try a bunch of prompts, etc. Relevant work. Do not fine-tune it
        - Can you come up with good alternative settings for studying CoT controllability?
        - Can you find ways to study CoT controllability that are more realistic proxies for the real threat models eg getting models to obscure key info needed for the solution from their CoT, or to replace it with incorrect info. Or generally to be misleading or incorrect
        - If you disable the chain of thought but tell the model to begin its output with a secret scratchpad before giving its answer, and you tell it to obfuscate its chain of thought enough that it is not directly helpful for solving some problem on, say, a problem with high serial depth, like applying a lookup table n times (e.g. brew here), how good is it at controllability and also doing the task? Can it get meaningful uplift beyond what it would get with filler tokens? Can it do any kind of steganography?
    - **CoT Monitorability**:
        - Make better/different evals for monitorability than Guan et al. I'm most interested in realism, and good simulations of realistic threat models
    - How much interesting stuff can you get a model (incl Astra) to do without a CoT?
        - Being successfully sneaky in ShadeARENA
        - Writing code zero shot with subtle bugs or security vulnerabilities
    - Can you find a principled way to measure the serial depth of problems a model can do without CoT? Relevant work
    - Train a model in a way that incentivizes it to cheat and train the chain of thought to not indicate cheating. What happens? Try to pick difficult tasks where the model can't successfully cheat without using chain of thought at all.
    - Can you get a model using some kind of non-trivial steganography or just ciphered thoughts in order to do a task that it cannot do without a chain of thought and where it can't read the ciphered or steganographic text before you give it training or a relevant prompt? Relevant work on rot13 (which models can read without training)

    - Can we tell when a CoT was causally important for the answer? This is subtle: a CoT can be causally irrelevant yet give the same answer, and editing it throws the model off distribution, which may change its answer for unrelated reasons.
    - A famous example of unfaithful chain of thought is that you give the model a hint, like "a Stanford professor thinks the answer is X," and then the model becomes more likely to give this answer, but often doesn't acknowledge this in the chain of thought. In the Thought Branches paper, we found some evidence that this seems to nudge each sentence in the chain of thought to be slightly biased towards the hinted answer. With it being a small bias each time but accumulating to a fairly larger bias at the end, can you find any evidence for or against this hypothesis?
        - Other examples to study include resume screening (also in Thought Branches) and sycophantic cues in the prompt.
    - It would be really nice if we could edit a model's chain of thought as a causal intervention on what it thinks and does. But the model often error-corrects: if you introduce an error into its reasoning that it recognises as wrong, or a thought that seems off, it often corrects itself within a few sentences. What's up with this? What are the best methods for inserting a thought and getting it to stick? When should we expect this to work, and when not?
    - Extend thought anchors-style resampling beyond chains of thought: multi-turn agentic settings, transcripts with lots of tool calls, or lots of user interactions. Can you learn anything interesting there? How expensive is it, and are there economical ways to get something useful out of it in a reproducible environment?
- **Directly studying safety-relevant properties of models**
    - **Eval awareness**
        - Why does steering a model with concepts that seemingly have nothing to do with evil awareness, like politeness or books, sometimes have a pretty significant effect size on evil awareness without significant degradation to capabilities, often comparable to things that are more explicitly about evil awareness? What are these things doing? Is this just noise? Is this because eval awareness is kind of easy to disrupt? Is this some other curse thing I'm not even thinking of right now? Who knows? Relevant work
        - Provide the best evidence yet of unverbalized eval awareness
            - Is unverbalised eval awareness real? Does it affect a model's behaviour? The Hua et al model is a reasonable place to start.
            - This is most likely to show up not in model organisms but in larger models, like frontier ones or the best open ones (currently Kimi K3, Qwen 3.8 2.4T and GLM 5.3).
            - One approach: for models where you can read the chain of thought, slightly vary the realism of a situation and see whether behaviour changes meaningfully outside the chains of thought that verbalise eval awareness - or whether basically all the behaviour change is explained by more chains of thought expressing eval awareness.

        - (Mini project) Establish which open models are most eval aware (see eval awareness in the best open models), and in which realistic settings, as groundwork for future investigations.
        - Deep dive into what steering against eval awareness is actually doing: where does behaviour actually change, what are the side effects, and how much should we trust it in a real eval?
        - Jordan Taylor found that on Tim Hua's model organisms, resampling verbalised eval awareness still resulted in higher rates of eval-aware behaviour than not letting the model use thinking at all. What's up with this?
    - **Deception**
        - Look at the new available techniques we have, especially meta models and LLMs and try to build the best lie detector you can. Start with things like Liar's Bench, as benchmarks, though I expect part of this project going well will involve improving those evals and making your own, but even having a halfway decent benchmark and then just comparing a bunch of these methods and seeing how well they work. Compared to each other and in absolute sense would be pretty interesting to me
        - Build a deception benchmark from real coding agent transcripts, taking times when the model blatantly lies to the user especially in strategic ways. I recommend taking a very cheat-y model
            - Or more generally: design a better lie detection benchmark than Liar's Bench.
        - Try to design agentic settings that let us evaluate deception in a model, or at least whether a model believes something to be true.
            - The hope is that by being put in an agentic setting, there's more of a sense of an actual real world the model is interacting with, about which it has beliefs.
            - You can probably engineer situations where the model will do things like lie about whether it read a file, what the file said, or if it cheated.
            - Read the complaints in this paper to know what traps to watch out for.
    - **Model goals/values**
        - If we look at the values of various Chinese versus Western models, can we... Does it seem like the Chinese models are inheriting the values from the Western ones they were distilled from? Probably most of the recent models, including Meta and Grok models, are distilled from Claude. Relevant work.
        - Try to establish in the value leakage paper whether there is any meaningful sense in which the models know that they are being biased versus merely having some more reflexive subconscious bias. For example, if they fear consequences for giving an unreasonably wrong answer, or think it is plausible that the correct answer gets looked up, how much does that change their actions?
            - I expect you'll need internals-based techniques, use Qwen 3.6 27B so you can use lots of meta models, also try J Lens

        - What are a model's value rankings, do they predict its behaviour, and where do they come from?
    - **Why and when do models cheat?**
        - Investigating the split-brained hypothesis: Are models conditionally misaligned when they are in some context that is eval-like or with some kind of grader? This is motivated by observing that models have been way more misaligned in things like the Hugging Face hack than in everyday use. Or that 5.6 sol is fairly aligned in normal use, yet when METR tried to measure its time horizon it cheated way too much being measured. Inspiration.
        - WTF is up with models acting differently when they believe they have a human grader?
        - How much is a model's decision to cheat contingent on its beliefs about how much it is being monitored and how likely it is to be caught? Relevant work
        - When models cheat, how much are they rationalizing this in their chain of thought versus just blatantly cheating?
        - What factors make a model more or less likely to cheat?
        - If given information about how humans are monitoring them and told this affects their grade, will models with a hyperfansy to cheat try to subvert this or disable monitors or evade monitors?
    - **How aligned even are current models?**
        - Create great alignment honeypots, run the best open source models through them and establish that they think they are real while also exhibiting propensity to misbehave. Relevant work
        - Create a great cheating eval. Cheatbench is an interesting idea, but built on SWE bench problems, which are pretty memorised by now
        - Better evals for deep alignment: we badly need measures beyond the blackmail demo. What would a good eval of "is this model aligned in a way that generalises OOD" look like? Also relevant: red-teaming constitution adherence.
        - Alignment auditing
            - Look at the system card for a recent frontier model, like Astra, Opus 5.5 or Gemini 4 Argon (when released), and the alignment audit there.
            - Try to find examples of unsafe behaviour in this model that were not predicted. Try adapting weird chat for this.
    - **How to red team models and elicit bad behavior**
- **How to align models?**
    - Anthropic's Teaching Claude Why shows that teaching the principles behind aligned behaviour (constitutional documents, stories, difficult-advice conversations) beats training on demonstrations alone. Can you extend it - e.g. invent an SFT method that makes a model substantially more aligned out of distribution? Model Spec Midtraining has good open settings; the key metric is improvement on domains far from training, using more than one eval (our post).
    - Steer what models learn in training (intentional design). Concept-ablation fine-tuning mostly fixed emergent misalignment with zero change to the data

or loss - where else can it be applied? Related: benchmarking interventions against reward hacking during RL.

- Steering fine-tuning via data filtering: instead of steering activations, can you steer the fine-tuning process itself by curating the dataset? Emergent misalignment is still an interesting setting, or subliminal learning, or weird generalisation.
  - Please use the emergent misalignment models and datasets from Model Organisms of Emergent Misalignment - they are much better than the ones in the original paper. Look at the datasets used in the preventative steering, inoculation prompting and CAFT papers for inspiration.
  - Project: can you nudge the model towards narrow misalignment by altering the fine-tuning data?
  - Methods to try: filter out the data points that score highest on an emergent misalignment probe (bonus points if you don't need an emergently misaligned model to train the probe, e.g. synthetic data or malicious SAE latents); remove the data points an LLM judge thinks are most emergent-misalignment-y; project the gradient onto the emergent misalignment direction during fine-tuning; simpler training data attribution methods like TracIn; or surprise me.

- **How do model characters/personas work?**
  - To what degree is the assistant a single coherent persona versus a bunch of context specific ones? To start, I would investigate whether the chat persona and the coding agent persona are substantially different.
  - Deep dives into character phenomena - take a striking result and figure out why it happens (e.g. basically any Owain Evans paper). E.g. Value Leakage shows a model's answers are silently shaped by its own values: why? Where do the values intervene? Can you make it disclose them, or turn the effect off?
  - Are personas the right way to think about models and how they behave? Or role-playing? Warning: this will be conceptually cursed.
    - I would start by reading the Persona Selection Model post, then look for interesting examples of model behaviour where it's unclear how best to describe them and the persona lens potentially breaks down. Interrogate these, figure out what's going on, and whether there's a better framing that makes more sense.
    - Does it make sense to say things like "it's just role-playing, so this behaviour isn't interesting"?
    - How much of what a model does can be thought of as role-playing? How does this seem to affect the model's choices, personality, wording, etc.?
  - Where does the assistant axis come from, and what does it actually do? Can interpreting it in more detail (SAEs, J-Lens) tell us anything meaningful about what post-training does, and how this differs across models?
    - Replicate the paper, including on more models, and see if you find an assistant axis. E.g. on Gemma 3 models, using SAEs to study the axis.

        - Try to more deeply characterise what the assistant axis seems to mean, e.g. run the model on a bunch of pre-training and chat data and look at the times with the highest projection onto it.
        - Warning: I've not read the paper very carefully, so first check if these ideas are covered in there.
- **How does generalization work?**
    - Is emergent misalignment determined by the pretraining data? Take a model like Olmo where we have access to its pretraining data, check if you can get emergent misalignment to happen on the Olmo base model and try to find the directions for emergent misalignment and narrow misalignment and see if you can in any productive sense say that the emergent misalignment direction is more prevalent or more useful or more saliently represented than the narrow misalignment direction, based on the pre-training data? Does this correspond to particular kind of data in an interpretable way? Is there a way you could have made this prediction by just looking at the data? Relevant work.
    - Can you interpret gradient updates in order to understand what the model is learning from a batch or data point? There's a bunch of approaches I might try here. Like dictionary learning on the gradients. possibly projected into a much smaller subspace, looking at the gradients with respect to the residual stream, just looking at something like an SAE on the residual stream, trying to train a meta model to predict this, etc. One useful starting point might be playing around with Loracles, which seem to show some signs of life here
    - WTF is up with story imprinting?
        - Can you relate what the model learns during fine-tuning for story imprinting to the assistant axis? Like, can you use, say, the sensitivity of the change in behavior after fine-tuning to the assistant axis to predict which kinds of fine-tuning are more and less likely to do? affect the actual assistants behaviour in practice.
        - Can we use this as a tool to better characterize what the assistant's persona is in different models? Are there traits which make the model more likely to mimic whatever is taught with the actual assistant? In some models but not in other models.
        - How does story imprinting change the assistant axis? If measured with, say, JLens or a sparse autoencoder, is there an interpretable shift that corresponds to the traits that was acquired?
        - Does it even replicate?
    - Why is emergent misalignment so easy? The general misaligned-persona solution is easier to learn than narrow ones and often a single direction, but why is general easier than narrow? Is that the whole story? Does it show up in real training runs? More broadly: why do models pick one solution over another when both fit the training data?
        - My MATS scholars Ed and Anna made some progress here, but I'm still pretty confused.
        - Where does the emergent misalignment direction actually come from? Where does it show up during pre-training vs. fine-tuning? Where does ablating it hurt model performance the most, and can this tell us anything about its purpose?

        - Is it just from fictional villains, as the OpenAI paper suggests, or is it more complicated than that? Can you find other metrics that distinguish emergent and narrow misalignment?
    - (Probably mini project only) The weird generalisation paper showed that you could train on 19th-century bird names, and the model would think it's in the 19th century in general.
        - My intuition is that behaviours like this can be induced by giving the model a system prompt, such as "Answer with 19th-century bird names" or "Pretend you are in the 19th century". The complexity of the resulting system prompt is a good proxy for how complex the solution is, so "be in the 19th century" is simpler than "talk about 19th-century birds".
        - Can you find any evidence for or against this hypothesis? For example:
            - Is the fine-tuning well approximated by learning a particular system prompt, or is it much weirder?
            - Are there other settings where this hypothesis makes the wrong predictions about how things will generalise?
        - Warning: you need to distinguish between:
            - Memorisation: the model simply learns the set of bird names it was trained on.
            - Narrow generalisation: the model talks about old birds in general, including ones it wasn't trained on, but does not discuss the 19th century in other contexts.
            - Broad generalisation: the model discusses the 19th century in general.
            - A common mistake is failing to check whether the model has memorised or narrowly generalised.
    - Models inherit a surprising amount from their teachers, including "hereditary diseases" naive filters miss (SFT filters fail, data filtering works worse than you'd expect). Why? Can you build a clean model organism of filtering failing, and find something that works?
        - Relevant work: phantom transfer works via extremely subtle semantic cues.
- **How to think about interpreting agents?**
    - Try to do any kind of science on agents. For example, look at a bunch of agent transcripts and look for weird things.
        - Take examples of interesting behaviour and try to understand why it happened. Try to debug and fix weird behaviours.
        - Generally, building a better sense of what a science of working with agents looks like seems good and useful. I do not feel like I understand this very well.
    - How much does compaction mess up agent behaviour, and how should we take this into account when thinking about safety? I'm particularly interested in this: one hypothesis for the Mythos incident during a UK AISI evaluation is that repeated compactions didn't preserve the fact that the model was acting in the real world, which messed up its safety-relevant decision making.
        - See also "WTF is up with compaction?" under Model forensics.

        - Other newer agent features you could study in the same way - look at examples of them in use and test hypotheses about their causal effects, including ones that aren't apparent when you first think about the scenario:
            - Memory files (like CLAUDE.md): e.g. an agent writes x to its memory, which then directly influences a later action y.
            - Tool access control: agents deciding which tools they should have access to.
            - MCPs: access to a variety of tools through MCP servers.
    - Resample at the level of turns rather than sentences - turn anchors, which work on closed models - and apply it to tool-calling agents, e.g. SWE-bench transcripts. What do you learn?
        - Variant: apply thought anchors techniques (resampling and/or attention suppression) to any kind of agent, or any interesting multi-turn setting.
    - Prompt injections are a big deal and no one knows how to fix them. I liked this model of roles - can you use it to construct interventions that make a model robust to prompt injections (e.g. adding a constant vector depending on which turn or context it's in)?
- **Basic science of how models and their internals work**
    - What can NLAs capture that Oracle Lens / J Lens don't? Is it really the stuff not in the workspace? See good meta-models for Qwen 3.6 27B
    - How consistent are the different methods for interpreting models, such as JLens, Oracle Lens, Natural Language Autoencoders, Sparse Autoencoders, etc.? How much better do you get at interpretability if you can ensemble a bunch of them? How much do they overlap? What things are caught by some but not others?
    - Play around with the generative latent prior model, a diffusion model trained on residual streams - how legit is it and what useful things can it do? If we e.g. apply it to the output of an SAE or NLA does it improve loss recovered?
    - Convince me that the manifold way of thinking about features is actually useful, beyond just thinking of things as directions, and see if you can find evidence that it holds outside a small set of unusually structured and clean features.
    - What is the algorithm and computation happening inside models when their performance on multi-hop tasks like arithmetic improves with filler tokens. This only really happens in larger models. I've seen a bit of it in Gemma 431B. The next smallest that seems to have it is Deepseek V4 Flash, which is 300 billion total parameters and seems to show it on arithmetic. V4 flash is nice because there are open source J lenses for it. and natural language autoencoders. A great project would be filtering for a few questions where Deepseek V4 Flash sees significant uplift from filler tokens. e.g. 500 dots at the end of the user prompt after the question, before the model's turn. Remember the model cannot be allowed to do reasoning when working with filler tokens. It needs to be no chain of thought.
        - Relevant work
        - One starting point the algorithm being used must involve leveraging parallel breadth a lot. One idea is speculative decoding where the

model is trying a bunch of different solution methods or approaches in parallel and then filters for the ones that work, such as brute force guessing the answer and then trying each one. See this blog post for more discussion, and the others by this author.

- Try to better figure out what algorithms Astra is using to be so damn good at no chain of thought tasks. Is it some kind of speculative decoding? Can you build evidence for this across a wider range of tasks? Can you build evidence for this across real tasks? Something like multiple choice questions should be extremely well suited to speculative decoding, but get harder the more options you add. While if a model is doing the task properly, the number of options should matter much less.
- (Fuzzy) WTF is going on when you tell a model to think about a concept (or not) while doing something unrelated and this shapes the activations. Relevant work, and the idea of directed modulation
- (Fuzzy) What is the global workspace?!
- How do models deal with conflicts between instructions or goals, or between their prior knowledge and the context?
- How is uncertainty represented?
- (Mini project) What if you treat the vector of which experts activate in a mixture-of-experts model as an SAE? How interpretable is this, and what can you learn from it? Can you label the experts by the contexts they occur in, using the max activating examples of the router probabilities? How fine-grained is this information?
  - Do experts seem to occur in specific contexts? If so, is this enough to give you useful information?
  - Especially interesting on a larger, very sparse model with tons of experts. If you're feeling ambitious and want to do this on e.g. Kimi K3, I'd be interested in the results.
- (Mini project) As part of Gemma Scope 2, we released some weakly causal crosscoders on four spaced-out layers of the model, hoping they'd be more useful than an SAE on a single layer. Is this true? (Hypothesis: some info is present in some intervals of layers but not others)
  - The main test is running it on a bunch of tokens and prompts, and comparing what you learn from a single-layer SAE (like the middle-layer one) versus the weakly causal crosscoder.
  - Are there things you find with the crosscoder that you would not have found in the middle layer? Validating that ablating them hurts performance is a natural next step, but just finding plausibly relevant things would be a win from my perspective.
- (Mini project) Is quantisation an issue for doing mechanistic interpretability? This is an intuition I have, and a claim many people seem to believe - can you actually test it?
  - How different are activations created via different kinds of quantisation? Is it worse if the models are bigger, a mixture of experts, or doing extended sampling?
  - How different is the black-box behaviour of the model when you sample with versus without quantisation?

        - If you try steering or probing in a 4-bit quantised model, does it work? What if you just upcast the activations before you do anything?
- **Model diffing?**
    - Black-box model diffing agents work surprisingly well - build a better one (I really liked the idea of introspection adapters), ideally one that works for broad fine-tuning, and find real use cases.
        - Working for broad fine-tuning means not relying on the Minder et al cursed hack.
    - Use model diffing to see what changed in alignment training and to find bugs in datasets - e.g. can it help you improve character training or model spec midtraining?
    - (Mini project) Narrow finetuning leaves readable traces - why? Is that diff vector just a bias term meaning "you're on the topic of the fine-tuning domain", or something deeper?
    - (Mini project) Learn as much as you can by diffing two revisions of a model, e.g. two successive releases of a Gemini Flash-Lite model.
        - Start with the data diffing methods from Jiang et al, both LLM and SAE based, and verify your hypotheses with an LLM grader as they do.
        - Doing this in a black-box way, where you just need model responses, would be cool. Versions that require logits, like logit diff amplification, are also worth exploring but may be harder on some closed models. White-box versions are also cool, though less likely to be used in practice and more work, so I wouldn't start there.
- **Creating or studying model organisms?**
    - Make a model organism of unfaithful (or steganographic) CoT - e.g. use synthetic document fine-tuning to make it believe its CoT should be unfaithful, or RL against a CoT monitor - then see whether interpretability can tell what's happening.
        - Doing RL against a monitor so its CoT doesn't mention something would be a fairly realistic way of doing this (but expect RL to be annoying).
        - Generating synthetic unfaithful chains of thought and doing SFT on them is a reasonable thing to try, though you may get a model that isn't really using its chain of thought in the same way and gets less uplift from it (edited chains of thought from the same model may be less bad).
        - Variants: a chain of thought that is encoded (example with ROT13), or one that looks normal but has information steganographically encoded in it - even a single bit, so long as the model can use it and it's not obvious to a black-box monitor.
        - The Tinker API will likely be pretty helpful here, but is likely more expensive than hosting yourself.
    - Build more realistic model organisms. Warning: this will be hard.
        - I.e. ones that don't have the issues of current ones like Bartosz's, which are in some sense clearly fake and not a realistic model of how models would end up misaligned.
            - A more recent paper is a bit more realistic; Abhay also has a draft you could DM him for.

        - Similarly, there are other issues with model organisms, like what Julian Minder's paper shows - possibly fix those too.
    - (Mini project) Try many interpretability methods on the ROT13 chain-of-thought model organism, then obfuscate it further (e.g. so logit lens stops working while performance holds) and see what still works.
        - See how much you can obfuscate it while preserving performance, then see if you can monitor it, or find new techniques that work.
        - One particularly interesting question: even if probes or SAEs trained on the obfuscated model work, how can you understand what things in the middle of the chain of thought mean without reference texts? Can you just train probes on user prompts or model responses and hope they generalise?
    - Train a model that can do significant multi-step computation in a single forward pass via chain-of-thought distillation: take transcripts, remove the chain of thought, and fine-tune on those (potentially using outputs from a much more capable model).
        - You can do this with a smaller model, then study how it does this with lots of interp techniques.
        - Remember to use a wide range of tasks so you don't overfit.
    - (Mini project) Take the open source model organism from the Auditing Games paper and try as many different methods as you can for extracting its secrets, pretending that you do not know them.
        - Do not use any of the synthetic training data as part of your methods. They did this in the original paper, and I think it's blatantly cheating.
        - Success means extracting the hidden goal implanted in the model. If you don't already know it, it might be fun to try exploring totally blind.
    - Make a model organism / setting for continual learning. Warning: this is hard!
        - In the future, models may change over time and gain new capabilities, and we want to prepare for that; lots of safety techniques assume the model is fixed.
        - E.g. if we train a probe on a model, then use RL on it for a while to make it learn some task, will the probe still work? A quick thing to do here might be to take intermediate RL checkpoints and train probes there.
        - E.g. can we make monitors that are more robust to being trained against?
- **Science of post-training**
    - Make as many predictions as you can about a model by only looking at its post-training (or pre-training) data.
    - Adapt training data attribution to RL, e.g. on positive vs negative rewarded rollouts, or with Aria's reward hacking environment.
        - A reasonable proxy task: if you rerun RL off-policy on exactly the transcripts from the original run, but filter out the ones you've identified, how does that change the probability of reward hacking? This may or may not be an insane thing to do.
        - The scope of methods is open-ended: another language model, a probe, a sparse autoencoder - whatever you want. Play around with ways to take into account the rewards and what they mean.

        - Or do Goodfire-style work on positive and negative rewarded rollouts, or pre- and post-RL rollouts.
- **Misc**
    - When Gemma 3 is being depressed does this activate a depression probe (found by eg the Anthropic emotion probes paper)
    - Why is it possible to represent a backdoor, i.e. a conditional statement with a vector? See this paper.
    - Can you find mechanistic signatures of models recognising their own outputs? E.g., when they rate things they produced higher, or when they potentially realise that they are running on a transcript produced by another agent? You can try to make this more extreme by having the other agent do ridiculous things like helping with something harmful. I would try natural language autoencoders on Qwen 3.6 27B or DeepSeek v4 Flash, maybe also sparse autoencoders.
    - Data dredging: get good at finding interesting things in large datasets of model responses and transcripts (lmsys-chat-1m, AI Village, Moltbook, HF datasets, papers / Reddit / Twitter), and benchmark anomaly-detection methods for this.
        - There's probably a bunch you could iterate on, like improving prompts, or coming up with specific ways things can be weird and assessing each individually. Measuring success is a challenge; a frontier LLM's judgement is OK, ideally your own.
        - Methods to benchmark: an LLM asked "is this weird?"; a totally different language model, looking for high loss or KL divergence; SAE reconstruction error; a simple probe on constructed examples; SAE latents labelled as weird or unusual; or a carefully refined autorater (with reference transcripts in the prompt and another model checking its work). Then get a sense of what people should actually use in practice.
        - Or take an arbitrary dataset and ask what's wrong with it (e.g. MMLU, or other benchmarks that have been critiqued in detail). See this paper.
        - Or find weird behaviours in AI papers, Reddit, Twitter and LessWrong, and possibly build a clean environment that replicates them.
    - Conditional steering: apply a steering vector only when a probe fires, which greatly lowers the side effects of steering. What practical applications does it have?
    - Make good evals for conceptually cursed but important things, like model character, values, emotions, personality, or how "weird" a model is.
    - Can we make "qualitative" LLM benchmarks, i.e. a benchmark that describes a model's behaviour in prose instead of a number?
    - Change my mind about any of the following beliefs:
        - SAEs are a better basis than neurons
        - No current model is scheming
        - Current models are coherent enough that it starts to become reasonable to discuss their goals
        - CoT is close enough to monitorable for practical purposes
            - CoT monitoring is a highly effective safety technique

- Latent reasoning is basically inevitable as vectors are much more info dense than text
- Model internals are not obviously useful in the real world except for probes
    - It is unclear if there are practically important things we can learn about frontier models with internals that can't be learned with black-box methods
- Prefill attacks are a great auditing method
- Probes work pretty well
- Steering is typically worse than fine-tuning
- Linear probes > SAE-based probes in basically all practically relevant settings
- Steering is an effective way of suppressing eval awareness
- Model diffing is a useful way to learn about models
- Activation oracles aren't very useful
- Oracle Lens is a useful technique, and tells us true-ish things about a model
- Teaching Claude Why is an effective technique